



南開大學
Nankai University

College of Computer Science
Natural Language Processing Lab Report

Assignment 1: Text Classification

Name: Li Jiapu

Student ID: 2314007

Major: Computer Science and Technology

October 8, 2026

Contents

1	Objectives	2
2	Experimental Environment	2
3	Datasets and Experimental Setup	2
3.1	Datasets	2
3.2	Data Split	3
3.3	Evaluation Metrics	3
3.4	Common Training and Model Selection Procedure	3
4	Implementation	4
4.1	Task 1: Bag of Words	4
4.2	Task 2: Word Embeddings	4
4.3	Task 3: BERT Fine-tuning	5
4.4	How the Experiments Are Run	6
5	Results	6
5.1	Overall Results	6
5.2	Per-class Results	7
5.3	Hyperparameter Selection on the Validation Set	7
5.4	BERT Training Progress	8
6	Analysis and Comparison	8
6.1	Overall Performance and the Role of Macro-F1	8
6.2	Binary Bag of Words versus Word Frequency	8
6.3	Averaged Word Embeddings	8
6.4	The Performance of BERT	9
6.5	Summary of the Methods	10
7	Reflections	10

1 Objectives

This assignment compares three families of text representations on the same news classification task: traditional bag-of-words models, averaged word embeddings, and fine-tuning a pre-trained language model. The specific goals are:

1. Understand the difference between the Binary Bag of Words and Word Frequency representations;
2. Master the basic workflow of a text classification experiment: data splitting, tokenization, feature construction, classifier training and evaluation;
3. Use Logistic Regression as a common classifier to compare the effect of different text representations;
4. Learn to use pre-trained GloVe vectors, and train Word2Vec with gensim on AG News and on NYT;
5. Become familiar with fine-tuning BERT-style pre-trained models;
6. Evaluate models with Accuracy and Macro-F1, and compare and analyze the performance of the different methods.

2 Experimental Environment

Item	Description
Hardware	GPU: NVIDIA A40 with 48GB of memory, driver version 550.54.14
Software	Python 3.10, PyTorch 2.4.1+cu118, transformers 4.46.3, scikit-learn 1.5.2, gensim 4.3.3, nltk 3.9.1, numpy 1.26.4, pandas 2.2.3
Pre-trained re-sources	The GloVe file <code>glove.6B.100d.txt</code> , which contains 400,000 words with 100 dimensions each; the pre-trained model <code>google-bert/bert-base-uncased</code>
Running time	14 min 31 s for the three tasks run one after another: about 3 minutes for Task 1, 8 minutes for Task 2 and 3 minutes for Task 3, of which BERT training itself took only 53 seconds
Code repository	https://github.com/c20201057/nlp/tree/HW1

Table 1: Experimental environment

3 Datasets and Experimental Setup

3.1 Datasets

The **NYT dataset** is stored in `nyt.csv`. It contains 11,519 news articles with two columns, `text` and `label`, and three classes: business, politics and sports. All text is already lowercased. Split on whitespace, an article has about 638 words on average, the median is 672 words, the shortest article has 33 words and the longest has 5,134 words. The class distribution is highly imbalanced: sports accounts for 75% of the data, while business and politics each account for about 12.5%. The data contains 72 exact duplicate records. The assignment does not ask for deduplication, so the original data is kept unchanged.

The **AG News dataset** is stored in `ag.csv`. It contains 90,000 news articles with a single `text` column, about 39 words per article on average, and is only used to train Word2Vec embeddings.

3.2 Data Split

As required by the assignment, the NYT data is shuffled with random seed 42 and split into training, validation and test sets in the ratio 80%, 10% and 10%. Every experiment calls the same splitting function `load_nyt_splits()`, and the split indices are saved in `results/nyt_split.json`, so the results of different methods are directly comparable. Table 2 shows the class distribution of each set.

Set	business	politics	sports	Total
Training Set	1157	1134	6924	9215
Validation Set	124	152	875	1151
Test Set	148	165	840	1153

Table 2: NYT data split and class distribution

3.3 Evaluation Metrics

Let the test set contain N samples and let $\mathcal{C}$ be the set of classes. Accuracy is the fraction of correctly classified samples:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i]$$

Macro-F1 first computes the precision P_c , the recall R_c and $F1_c = \frac{2P_cR_c}{P_c+R_c}$ for every class c , and then takes the arithmetic mean over all classes:

$$\text{Macro-F1} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F1_c$$

Because sports makes up 72.9% of the test set, a trivial classifier that predicts sports for every sample already reaches an Accuracy of 0.7285, yet its Macro-F1 is only 0.281. Macro-F1 therefore better reflects how a model handles the two minority classes, business and politics. Both metrics are computed with scikit-learn's `accuracy_score` and `f1_score`, the latter with `average="macro"`.

3.4 Common Training and Model Selection Procedure

Tasks 1 and 2 both use scikit-learn's `LogisticRegression` with the `lbfgs` solver and `max_iter` set to 5000. The regularization strength C is chosen from 0.01, 0.1, 1, 10 and 100: a model is trained on the training set for each C , its Macro-F1 is measured on the validation set, and the model with the highest validation Macro-F1 is evaluated on the test set. The test set is used only once, for the final evaluation, and plays no part in any selection.

Listing 1: Excerpt from `common.py`: training LR and selecting C on the validation set

```

1 for C in C_grid:
2     clf = LogisticRegression(C=C, max_iter=5000, random_state=SEED)
3     if scaler:
4         clf = make_pipeline(StandardScaler(), clf)
5     clf.fit(X_train, y_train)
6     val = evaluate(y_val, clf.predict(X_val), labels)
7     if best is None or val["macro_f1"] > best[1]["macro_f1"]:
8         best = (clf, val, C)
9 clf, val, C = best
10 test = evaluate(y_test, clf.predict(X_test), labels)

```

All text is tokenized with `nltk.word_tokenize`. The tokenized text is cached in the `cache/` directory, so Tasks 1 and 2 share exactly the same tokens.

4 Implementation

4.1 Task 1: Bag of Words

The vocabulary is built from the training set only. After tokenization the training set contains $|V| = 128,343$ distinct words, and every document is represented as a $|V|$ -dimensional vector. Since almost all entries are zero, the vectors are stored as a sparse matrix. A dense matrix for the training set alone would need about $9215 \times 128343 \times 8$ bytes, which is roughly 9.5GB of memory.

The two representations differ only in whether word counts are recorded:

- **Binary Bag of Words:** $x_i = 1$ if the i -th vocabulary word appears in the document, otherwise $x_i = 0$;
- **Word Frequency:** $x = (c_1, c_2, \dots, c_{|V|})$, where c_i is the number of times the i -th vocabulary word occurs in the document.

The implementation uses `CountVectorizer` on the pre-tokenized token lists, and the `binary` argument switches between the two representations. Words in the validation and test sets that are not in the training vocabulary are ignored.

Listing 2: Excerpt from `task1_bow.py`: building the two bag-of-words representations

```
1 for name, binary in [("task1_binary_bow", True), ("task1_word_frequency", False)]:
2     vec = CountVectorizer(analyzer=identity, binary=binary)
3     X_train = vec.fit_transform(tokens["train"])
4     X_val, X_test = vec.transform(tokens["val"]), vec.transform(tokens["test"])
5     result = fit_lr_with_val(X_train, y["train"], X_val, y["val"],
6                             X_test, y["test"], labels)
```

4.2 Task 2: Word Embeddings

All three experiments use 100-dimensional word vectors. A document vector is the average of the word vectors of all valid words in the document:

$$e(d) = \frac{1}{n} \sum_{i=1}^n e(w_i)$$

A valid word is a word that appears in the vocabulary of the embedding being used; out-of-vocabulary words are skipped. In all three experiments every document contains at least one valid word, so no document ends up as a zero vector. The dense document vectors are standardized with `StandardScaler`, whose mean and variance are estimated on the training set only, before being fed to Logistic Regression.

Listing 3: Excerpt from `task2_embeddings.py`: computing document vectors

```
1 def doc_vectors(docs, lookup):
2     X = np.zeros((len(docs), DIM), dtype=np.float32)
3     for i, doc in enumerate(docs):
4         vecs = [lookup[w] for w in doc if w in lookup]
5         if vecs:
6             X[i] = np.mean(vecs, axis=0)
7     return X
```

Pre-trained GloVe. The 400,000 word vectors in `glove.6B.100d.txt` are loaded. GloVe 6B was trained on about 6 billion tokens from Wikipedia 2014 and Gigaword 5, and Gigaword is itself a news corpus. The token coverage on the NYT test set is 98.06%.

Word2Vec trained on AG News. The 90,000 AG News articles are tokenized and used to train a 100-dimensional Word2Vec model with gensim; the corpus has about 3.5 million words. The training parameters are listed in Table 3, and the resulting vocabulary has 43,870 words. Its token coverage on the NYT test set is 95.51%, the lowest of the three.

Word2Vec trained on NYT. The same parameters are used to train on NYT, giving a vocabulary of 57,775 words and a test-set token coverage of 98.37%. To keep validation and test text out of representation learning, only the 9,215 articles of the NYT **training set** are used here, about 5.9 million words in total.

Parameter	Value
Model	Skip-gram, <code>sg=1</code> ; negative sampling, <code>negative=5</code>
Vector size	<code>vector_size=100</code>
Context window	<code>window=5</code>
Minimum count	<code>min_count=2</code>
Epochs	<code>epochs=10</code>
Randomness	<code>seed=42</code> and <code>workers=1</code> ; single-threaded training makes the result reproducible

Table 3: Word2Vec training parameters, identical for AG News and NYT

AG News articles are short, so each article is fed to Word2Vec as one sentence. The longest NYT article has about 5,000 words, which is below gensim’s limit of 10,000 words per sentence, so NYT articles are also fed one article at a time. To confirm reproducibility, the complete set of experiments was run twice on the A40, and the two runs produced identical values for every metric.

4.3 Task 3: BERT Fine-tuning

The model is `google-bert/bert-base-uncased` with a randomly initialized 3-way linear classification head on top of the [CLS] representation, which is what `BertForSequenceClassification` provides, and the whole model is fine-tuned. As required by the assignment, tokenization uses `max_length=64`, truncating longer inputs and padding shorter ones, and the model is trained for 3 epochs. The remaining hyperparameters are listed in Table 4.

Parameter	Value
Pre-trained model	<code>bert-base-uncased</code> , about 110 million parameters
Maximum sequence length	64
Epochs	3
Batch size	32, giving 288 steps per epoch and 864 steps in total
Optimizer	AdamW, learning rate 2×10^{-5} , weight decay 0.01, no decay on biases and LayerNorm
Learning rate schedule	Linear warmup over the first 10% of steps, then linear decay to 0
Other	Gradient clipping at 1.0; bf16 mixed precision; random seed 42
Model selection	The model at the end of epoch 3 is reported; no early stopping on the validation set

Table 4: BERT fine-tuning settings

Listing 4: Excerpt from `task3_bert.py`: the training loop

```

1 for epoch in range(1, args.epochs + 1):
2     model.train()

```

```

3  for input_ids, attention_mask, y in train_loader:
4      with torch.autocast("cuda", dtype=torch.bfloat16):
5          loss = model(input_ids=input_ids.to(device),
6                      attention_mask=attention_mask.to(device),
7                      labels=y.to(device)).loss
8      loss.backward()
9      torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
10     optimizer.step()
11     scheduler.step()
12     optimizer.zero_grad()
13     val = evaluate(y_val, predict(model, val_loader, device), labels)

```

4.4 How the Experiments Are Run

The script `scripts/run_all.sh` runs the three tasks one after another. Results are written to `results/` as JSON files, and the full output is written to `logs/run_all.log`. The BERT weights are downloaded to `models/bert-base-uncased` in advance and loaded from the local directory in offline mode at run time.

Listing 5: Excerpt from `scripts/run_all.sh`

```

1  export PYTHONNOUSERSITE=1
2  export HF_HUB_OFFLINE=1 TOKENIZERS_PARALLELISM=false
3  exec > >(tee logs/run_all.log) 2>&1
4  "$PY" -u src/task1_bow.py
5  "$PY" -u src/task2_embeddings.py
6  "$PY" -u src/task3_bert.py --model models/bert-base-uncased

```

5 Results

5.1 Overall Results

Table 5 reports the results of the six experiments on the NYT test set, which contains 1,153 articles. The column “Errors” is the number of misclassified test articles derived from the Accuracy.

Task	Method	Selected C	Accuracy	Macro-F1	Errors
Task 1	Binary Bag of Words + LR	0.1	0.9870	0.9712	15
	Word Frequency + LR	100	0.9879	0.9724	14
Task 2	GloVe 6B-100d + LR	0.1	0.9922	0.9826	9
	Word2Vec trained on AG News + LR	0.1	0.9844	0.9653	18
	Word2Vec trained on NYT + LR	0.01	0.9879	0.9725	14
Task 3	Fine-tuned BERT-base-uncased	–	0.9792	0.9530	24

Table 5: Accuracy and Macro-F1 on the NYT test set

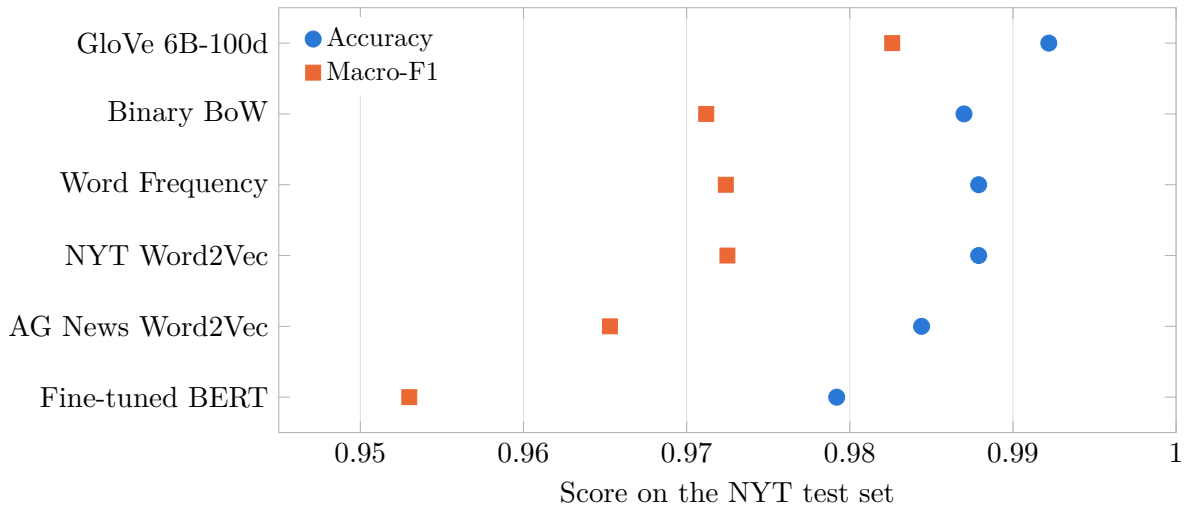


Figure 5.1: Accuracy and Macro-F1 of the six methods, ordered by Macro-F1 from low to high; the x-axis starts at 0.945 to make the differences visible

5.2 Per-class Results

Table 6 gives the F1 of each class on the test set. Every method reaches an F1 close to 1 on sports, so the differences in Macro-F1 come almost entirely from the two minority classes, business and politics.

Method	business F1	politics F1	sports F1
Binary Bag of Words	0.9550	0.9610	0.9976
Word Frequency	0.9550	0.9641	0.9982
GloVe 6B-100d	0.9763	0.9726	0.9988
Word2Vec trained on AG News	0.9521	0.9461	0.9976
Word2Vec trained on NYT	0.9553	0.9639	0.9982
Fine-tuned BERT	0.9324	0.9297	0.9970

Table 6: Per-class F1 on the test set

5.3 Hyperparameter Selection on the Validation Set

Table 7 lists the validation Macro-F1 for each value of C ; the selected C is shown in bold.

Method	$C = 0.01$	$C = 0.1$	$C = 1$	$C = 10$	$C = 100$
Binary Bag of Words	0.9661	0.9760	0.9711	0.9745	0.9720
Word Frequency	0.9619	0.9611	0.9627	0.9658	0.9756
GloVe 6B-100d	0.9545	0.9597	0.9568	0.9497	0.9438
Word2Vec trained on AG News	0.9581	0.9625	0.9586	0.9598	0.9514
Word2Vec trained on NYT	0.9677	0.9645	0.9556	0.9467	0.9395

Table 7: Validation Macro-F1 under different regularization strengths

5.4 BERT Training Progress

Epoch	Training loss	Validation Accuracy	Validation Macro-F1	Time
1	0.2814	0.9705	0.9277	24.1 s
2	0.0496	0.9713	0.9297	14.2 s
3	0.0182	0.9739	0.9369	14.4 s

Table 8: Training loss and validation results of BERT after each epoch

The first epoch takes longer, probably because it includes the initialization overhead of CUDA and its compute kernels. The training loss falls from 0.28 to 0.018, while the validation Macro-F1 only rises from 0.928 to 0.937: the model fits the training set very well, but the gain on the validation set is limited.

6 Analysis and Comparison

6.1 Overall Performance and the Role of Macro-F1

All six methods reach an Accuracy above 0.979, and the best and the worst differ by only 15 test articles. The three news topics are clearly separated; sports in particular has a large vocabulary of its own, such as team names, scores and competitions, so even the simplest bag-of-words model achieves very high accuracy. In this situation Accuracy is dominated by sports, which makes up 72.9% of the test set, and it hardly separates the methods. Macro-F1 gives the three classes equal weight and better exposes the differences on business and politics. For example, BERT and Binary BoW differ by only 0.8 percentage points in Accuracy but by 1.8 percentage points in Macro-F1.

Business and politics have only 148 and 165 test articles respectively, so a single misclassified article changes the F1 of its class by about 0.3 percentage points. The Macro-F1 gap of 0.0001 between Word Frequency and NYT Word2Vec, and the one-article gap between Binary BoW and Word Frequency, should therefore not be read as one method being better than the other; the methods in each pair perform about equally well.

6.2 Binary Bag of Words versus Word Frequency

The two bag-of-words representations produce almost the same test results, with 15 and 14 errors respectively. An NYT article has about 640 words on average, so whether a word appears at all already carries most of the information needed to identify the topic. Word counts add how often a word is repeated, but the most frequent words are mostly function words, which contribute little to topic classification.

The main difference between the two lies in the selected regularization strength: Binary BoW works best at $C = 0.1$, while Word Frequency works best at $C = 100$, the upper end of the grid. Count features span a much wider range of values than 0/1 features, since frequent words in long documents produce large feature values, so the same C constrains them differently. The selected C for Word Frequency lies on the boundary of the grid, which suggests that an even larger C might still help. This experiment uses the same grid for every method and does not extend the search for Word Frequency alone.

6.3 Averaged Word Embeddings

GloVe is the best of the six methods. It makes only 9 errors on the test set, reaches a Macro-F1 of 0.9826, and its F1 on both business and politics is clearly higher than that of any

other method. Two reasons are likely. First, GloVe was trained on about 6 billion tokens of Wikipedia and Gigaword news, three orders of magnitude more text than AG News or the NYT training set, so its word meanings are learned more thoroughly, and it comes from the same news domain as NYT. Second, the 100-dimensional averaged vector places semantically similar words close together. Compared with a sparse bag of words with 128,000 dimensions, the classifier has far fewer parameters to learn, which makes overfitting less likely when business and politics each have only about 1,150 training articles.

Word2Vec trained on NYT beats Word2Vec trained on AG News, with 14 errors against 18. The two models use exactly the same training method and parameters, so the difference comes from the training corpus:

- **Domain match:** the NYT vectors are trained on the corpus of the classification task itself, so word co-occurrences match how words are used at classification time. AG News is also news, but it consists mostly of short wire stories whose sources and writing style differ from NYT;
- **Vocabulary coverage:** the AG News vectors cover only 95.5% of the tokens in the NYT test set, against 98.4% for the NYT vectors, so more words are dropped when averaging;
- **Corpus size:** AG News has about 3.5 million words and the NYT training set about 5.9 million, so the latter is larger.

Both self-trained embeddings fall behind GloVe, which shows that at this scale the size and quality of the pre-training corpus matter more than being in the same domain as the task. NYT Word2Vec performs on par with the two bag-of-words representations, which indicates that, given the same training data, a 100-dimensional averaged vector keeps essentially all of the information in the bag of words that is useful for classification.

Averaging word vectors also has clear limitations: it discards word order, and it mixes in many words that are unrelated to the topic. It works well on this task because topic classification depends mainly on which domain words appear in the text and is not sensitive to word order.

6.4 The Performance of BERT

BERT has the most parameters and the most extensive pre-training, yet it scores lowest in this experiment, with a Macro-F1 of 0.9530 and 24 errors. The gap comes mainly from business and politics, whose F1 scores are only 0.9324 and 0.9297. I believe the main cause is the truncation imposed by `max_length=64`:

- An NYT article has about 640 words on average, while 64 WordPiece tokens, after removing [CLS] and [SEP], hold only about 50 words. The model therefore sees less than the first 10% of each article, whereas all the other methods use the full text;
- Business and politics articles both frequently discuss governments, companies and policies, so the two may be hard to tell apart from the first few dozen words, and the information that separates them may appear later in the article. Sports articles open with very distinctive features such as teams and scores, so the F1 on sports stays close to 1;
- In Table 8 the training loss drops to 0.018 while the validation Macro-F1 is only 0.937. The model already fits the training set very well, which is consistent with the explanation that the input lacks information.

In addition, the validation Macro-F1 is still rising after 3 epochs, so the number of training epochs may also limit BERT, but the assignment fixes it at 3 and this experiment does not change it. The truncation explanation above is an inference from the article lengths and the per-class results; this experiment does not include a control experiment that verifies it directly. Possible controls are to restrict the bag-of-words or GloVe methods to the first 64 tokens of each article as well, or to raise BERT's `max_length` to 512, and then check whether the gap shrinks.

6.5 Summary of the Methods

Method	Strengths	Limitations
Bag of words	Simple to implement and interpretable; uses the full text; already very strong for topic classification	As many as 128,000 very sparse dimensions; cannot exploit relations between synonyms; out-of-vocabulary words are dropped
Averaged word embeddings	Low-dimensional and able to exploit word similarity; GloVe, pre-trained at large scale, performs best	Discards word order; depends on the training corpus and coverage of the embeddings, and embeddings trained on a small corpus help little
Fine-tuned BERT	The strongest context-dependent representations; fine-tuning finishes in under a minute	Limited by the maximum sequence length, so it sees only the beginning of long documents; far more expensive to compute than the other two families

Table 9: Comparison of the three families of text representations

7 Reflections

The main lesson of this assignment is that a model with stronger representational power does not necessarily perform better on every task; results have to be interpreted in light of the task and the experimental setup. On a three-class NYT task with clearly separated topics, full-text bag of words and averaged GloVe vectors are already very strong, while BERT, limited to 64 tokens and therefore to the beginning of each article, performs worst. Expecting BERT to win purely because of its size would have led to the wrong conclusion.

The second lesson concerns the choice of evaluation metric. With imbalanced classes, Accuracy is easily inflated by the majority class, and Macro-F1 together with per-class F1 is needed to see how a model handles the minority classes. At the same time, the minority classes have few test articles, so a difference of a few articles is not enough to rank two methods, and comparisons have to allow for this random variation.

The third lesson is the importance of a disciplined experimental procedure. All methods share one data split, hyperparameters are selected on the validation set only, and the test set is used once at the end; the NYT Word2Vec model is trained on the training set only, so no test information leaks into representation learning. Fixed random seeds, single-threaded Word2Vec training and saved run logs all serve to make the results reproducible and checkable. Running the complete set of experiments twice and obtaining identical results confirmed this.

Finally, turning environment setup, resource downloads and experiment runs into scripts, and pinning dependency versions in `requirements.txt`, makes the whole experiment repeatable on a new machine.